



G L O B A L R A I N

Practices for Secure Software Report

Table of Contents

DOCUMENT REVISION HISTORY	3
CLIENT	3
INSTRUCTIONS	3
DEVELOPER	4
1. ALGORITHM CIPHER	4
2. CERTIFICATE GENERATION	4
3. DEPLOY CIPHER	4
4. SECURE COMMUNICATIONS	4
5. SECONDARY TESTING	4
6. FUNCTIONAL TESTING	4
7. SUMMARY	4
8. INDUSTRY STANDARD BEST PRACTICES	4

Document Revision History

Version	Date	Author	Comments
1.0	08/18/2026	Tajiona Thompson-Hervey	

Client



Developer

Tajiona Thompson-Hervey

1. Algorithm Cipher

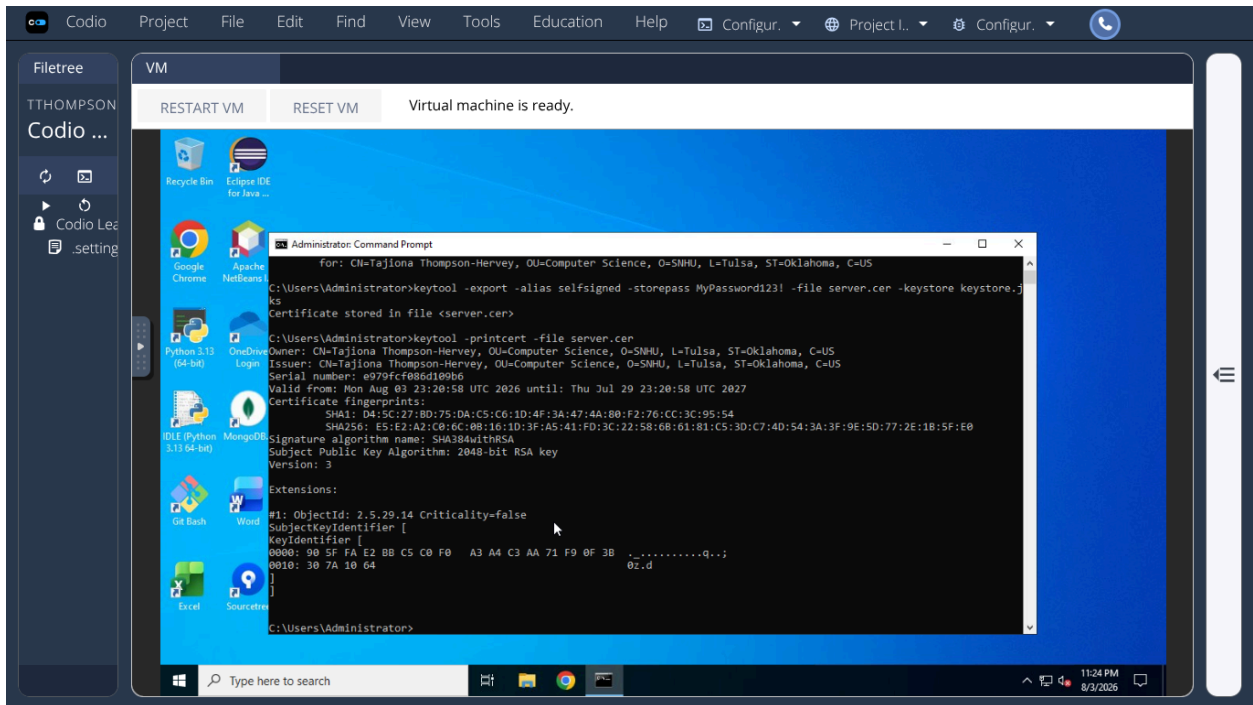
I selected SHA-256 as the cryptographic hash algorithm for Artemis Financial's checksum verification. SHA-256 is part of the SHA-2 family of cryptographic hash functions and produces a 256-bit message digest. A hash function takes input data and produces a fixed-length value that can be used to verify whether the data has changed. NIST describes secure hash algorithms as being used to generate message digests that can detect whether messages have been modified. SHA-256 is appropriate for the checksum requirement because the application needs to verify the integrity of data rather than decrypt the data. The resulting hash is not intended to be reversed back into the original string. The 256-bit output provides a significantly larger hash space than older algorithms such as MD5 and SHA-1, making SHA-256 a more appropriate choice for modern security requirements.

SHA-256 generates a 256-bit hash value. The input can be of varying length, but the resulting digest is always 256 bits. The hash is represented as a hexadecimal value when returned by the application. The large output space makes it computationally difficult to intentionally create two different inputs that produce the same hash.

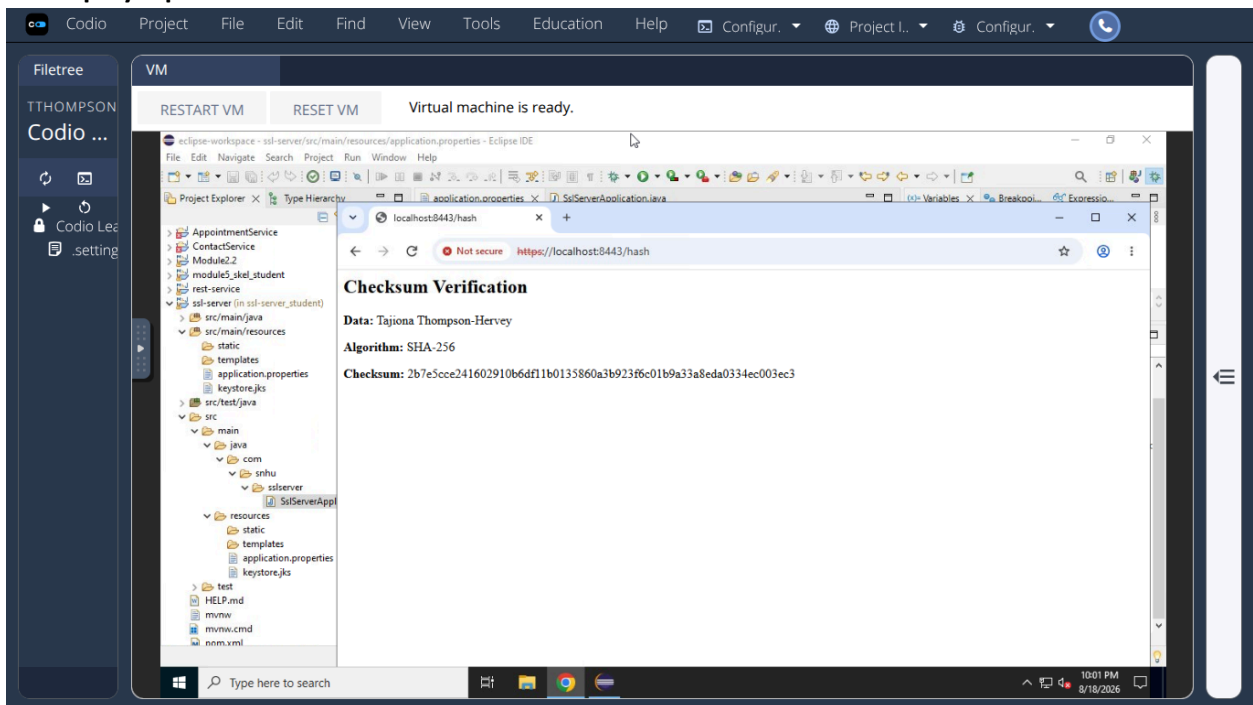
SHA-256 is a hashing algorithm rather than an encryption algorithm, so it does not use a secret encryption key to create the checksum. The certificate portion of the application uses an RSA public/private key pair. The RSA key pair and self-signed certificate provide a separate security layer for HTTPS communications. The Java `keytool` utility can generate a public/private key pair and store the associated certificate in a keystore.

Earlier hash algorithms such as MD5 and SHA-1 have known weaknesses and are no longer appropriate choices for new security-sensitive applications. SHA-2 algorithms such as SHA-256 provide stronger protection against collision attacks and remain widely used for integrity verification. For this application, SHA-256 provides an appropriate balance of security and compatibility.

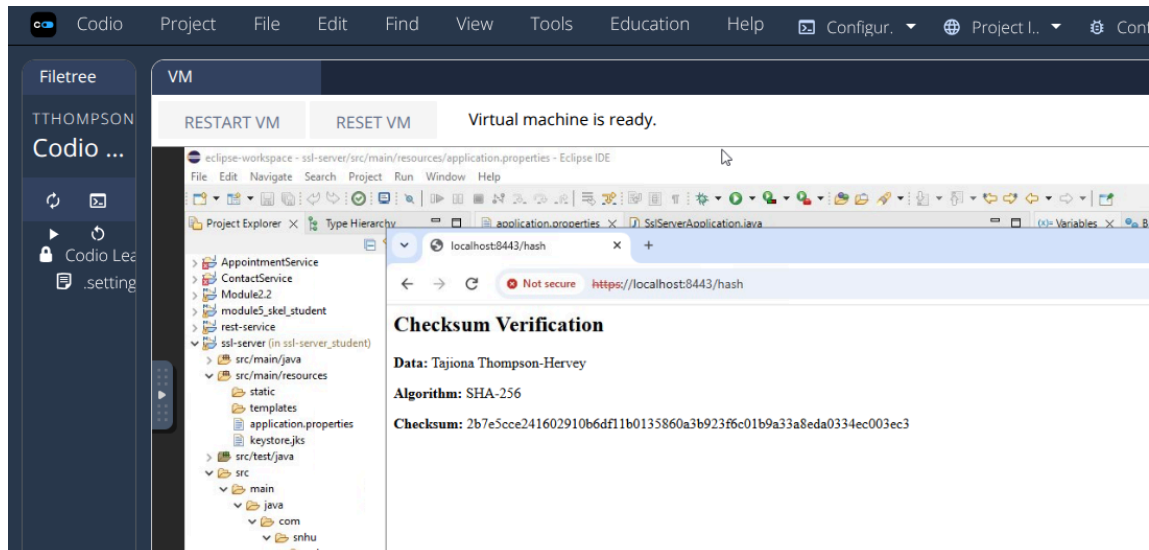
2. Certificate Generation



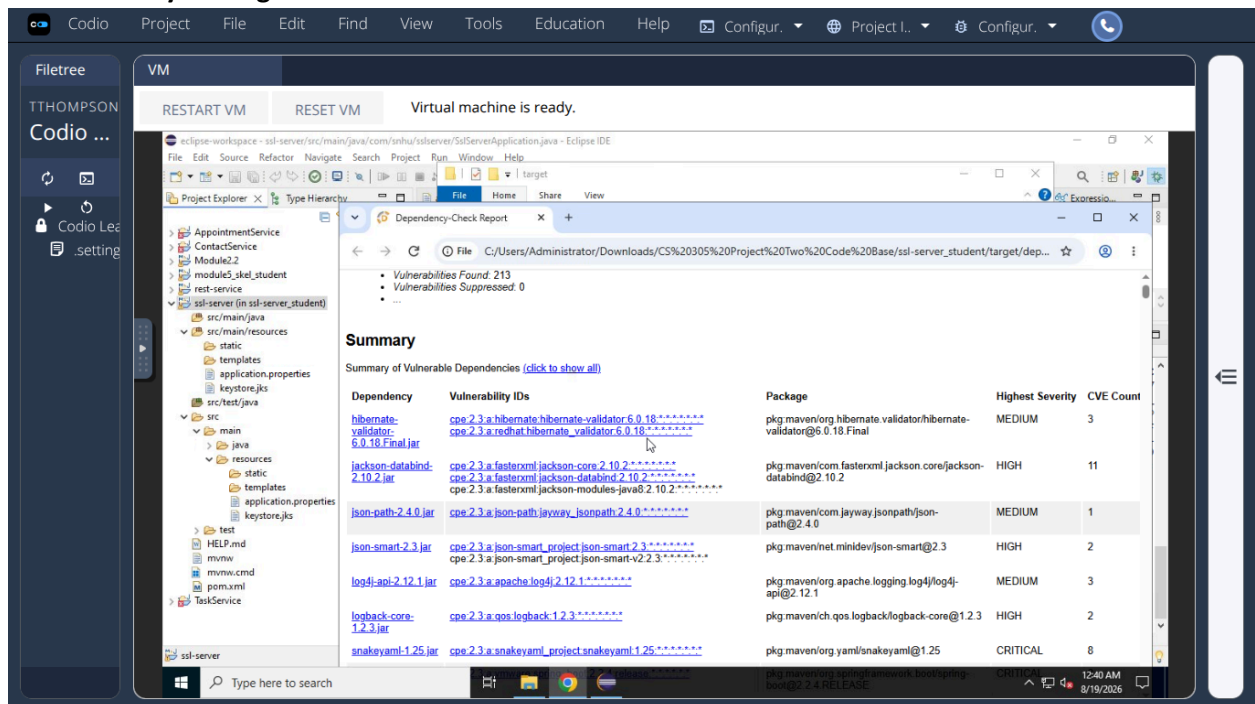
3. Deploy Cipher



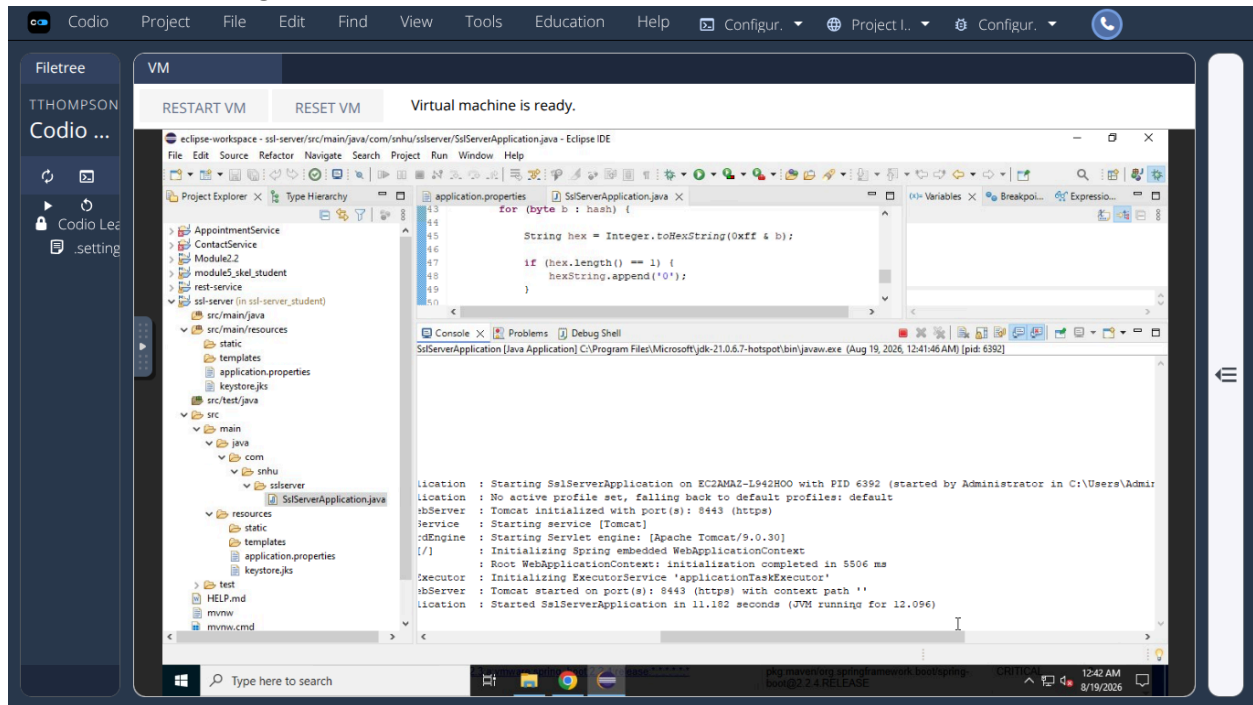
4. Secure Communications



5. Secondary Testing



6. Functional Testing



7. Summary

The application was refactored by adding multiple layers of security to protect communications and verify data integrity. SHA-256 was implemented to generate a checksum for the application's unique data string. A self-signed RSA certificate was generated using Java Keytool and incorporated into the application through a JKS keystore. The application was then configured to use HTTPS on port 8443.

The security changes provide separate layers of protection. The SHA-256 checksum helps detect changes to data, while HTTPS protects communications between the browser and application. Dependency-Check was also used as a secondary testing method to evaluate the application's dependencies for known vulnerabilities.

The refactoring process followed a security-focused development approach by implementing cryptographic hashing, certificate-based HTTPS communication, and dependency vulnerability testing.

8. Industry Standard Best Practices

Industry-standard secure coding practices were applied by using a modern cryptographic hash algorithm, implementing HTTPS for communications, using a certificate and keystore for secure server communication, and performing dependency vulnerability testing.

Using SHA-256 rather than an outdated hashing algorithm helps reduce the risk associated with weak cryptographic algorithms. HTTPS provides encryption for information transmitted between the client and

server. The Java Keytool utility provides functionality for managing public/private keys and certificates used for authentication and data integrity.

Dependency testing also provides an additional layer of security because applications rely on third-party libraries that may contain known vulnerabilities. OWASP Dependency-Check is designed to identify known vulnerabilities in project dependencies.

Applying these practices helps maintain the security of the existing application while reducing the risk that newly introduced functionality creates additional security weaknesses. Security testing throughout development can also help identify problems before software is released to users.